```
!nvidia-smi
```

```
Wed May  6 06:47:59 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   41C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
!pip install -q --force-reinstall "numpy==2.0.2" "pandas==2.2.2"
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.9/60.9 kB 3.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.2/19.2 MB 98.7 MB/s eta 0:00:00
```

```
import numpy as np
import pandas as pd
import torch

print("numpy:", np.__version__)
print("pandas:", pd.__version__)
print("cuda available:", torch.cuda.is_available())
```

```
numpy: 2.0.2
pandas: 2.2.2
cuda available: True
```

```
!pip install -q -U transformers accelerate peft bitsandbytes datasets trl sentencepiece scikit-learn pyyaml
```

```
import numpy as np
import pandas as pd

print("numpy:", np.__version__)
print("pandas:", pd.__version__)
```

```
numpy: 2.0.2
pandas: 2.2.2
```

```
!pip install -q --force-reinstall "numpy==2.0.2" "pandas==2.2.2"
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
DATA_PATH = "/content/drive/MyDrive/Colab Notebooks/valid instructions/valid_instruction_to_workflow_20000.csv"
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os

DATA_PATH = "/content/drive/MyDrive/Colab Notebooks/valid instructions/valid_instruction_to_workflow_20000.csv"

print(os.path.exists(DATA_PATH))
```

```
True
```

```
import pandas as pd

df = pd.read_csv(DATA_PATH)

print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
df.head()
```

```
Shape: (20000, 13)
Columns: ['id', 'dataset_name', 'source_reference', 'domain', 'complexity', 'user_role', 'instruction', 'output', 'step_count',
```

	id	dataset_name	source_reference	domain	complexity	user_role	instruction	
0	TRAIN_00001	valid_instruction_to_workflow	BPI_2019_P2P	procurement	simple	department_head	Kindly check the current status of purchase or...	p2p_proc_simpl
1	TRAIN_00002	valid_instruction_to_workflow	BPI_2019_P2P	procurement	simple	ceo	Finance asked me to raise a purchase order for...	p2p_proc_simp
							Could you fetch the	

Next steps: Generate code with df | New interactive sheet

```
df = df[["instruction", "output"]].dropna().copy()

df["instruction"] = df["instruction"].astype(str)
df["output"] = df["output"].astype(str).str.replace("\\n", "\n", regex=False)

print("Training rows:", df.shape[0])

print("\nSample instruction:")
print(df.iloc[0]["instruction"])

print("\nSample YAML output:")
print(df.iloc[0]["output"][:1000])
```

```
Training rows: 20000

Sample instruction:
Kindly check the current status of purchase order PO_2019_010002 for IT because of scheduled procurement and notify the responsi

Sample YAML output:
workflow_name: p2p_proc_simple_keyboards_00001_0028
description: Executes a valid BPI 2019 purchase-to-pay procurement workflow.
version: "1.0"
created_by_role: department_head
source_process_reference: BPI_2019_P2P
steps:
  - step_id: step_1
    action: fetch_record
    description: Fetch purchase order status
    parameters:
      table: purchase_orders
      filters:
        purchase_order_id: PO_2019_010002
        department: IT
    on_failure: retry_once
validation:
  rbac_required_role: procurement_officer
  estimated_risk_level: low
  requires_human_approval: false
  max_execution_time_ms: 2500
```

```
df_small = df.sample(n=min(1000, len(df)), random_state=42).reset_index(drop=True)
print(df_small.shape)
```

```
(1000, 2)
```

```
from sklearn.model_selection import train_test_split
from datasets import Dataset

train_df, val_df = train_test_split(
    df_small,
    test_size=0.05,
    random_state=42,
    shuffle=True
)

print("Train:", train_df.shape)
print("Validation:", val_df.shape)

train_dataset = Dataset.from_pandas(train_df.reset_index(drop=True))
val_dataset = Dataset.from_pandas(val_df.reset_index(drop=True))
```

```
Train: (950, 2)
Validation: (50, 2)
```

```
SYSTEM_PROMPT = """You are an Agentic Orchestrator for ERP workflow automation.
Convert the user's natural language request into a valid YAML workflow blueprint.
Use only the allowed ERP actions.
Return only YAML. Do not add explanations."""
```

```
def format_example(example):
    instruction = example["instruction"]
    output = example["output"]

    text = f"""<|system|>
{SYSTEM_PROMPT}<|end|>
<|user|>
{instruction}<|end|>
<|assistant|>
{output}<|end|>"""

    return {"text": text}

train_dataset = train_dataset.map(format_example)
val_dataset = val_dataset.map(format_example)

print(train_dataset[0]["text"][:1500])
```

```
Map: 100%                                                 950/950 [00:00<00:00, 15009.96 examples/s]

Map: 100%                                                 50/50 [00:00<00:00, 2547.07 examples/s]
```

```
<|system|>
You are an Agentic Orchestrator for ERP workflow automation.
Convert the user's natural language request into a valid YAML workflow blueprint.
Use only the allowed ERP actions.
Return only YAML. Do not add explanations.<|end|>
<|user|>
Kindly create a purchase order for keyboards and send it for department head approval because of quarter-end closure for the cur
<|assistant|>
workflow_name: p2p_proc_medium_keyboards_03801_3828
description: Executes a valid BPI 2019 purchase-to-pay procurement workflow.
version: "1.0"
created_by_role: department_head
source_process_reference: BPI_2019_P2P
steps:
  - step_id: step_1
    action: create_purchase_order
    description: Create purchase order requiring approval
    parameters:
      vendor_id: vendorID_4641
      item: keyboards
      quantity: 31
      amount_lkr: 225737
      currency: LKR
    on_failure: retry_once
  - step_id: step_2
    action: request_human_approval
    description: Request manager approval for purchase order
    parameters:
      approver_role: department_head
      context: Approve purchase order {{step_1.purchase_order_id}} for procurement request
      priority: normal
    depends_on: step_1
    on_failure: escalate_to_human
  - step_id: step_3
    action: send_email_notification
    description: Notify procurement after approval request
    parameters:
      recipient_role: procurement_officer
      subject: Approval requested for keyboards
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig

model_id = "microsoft/Phi-3-mini-4k-instruct"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)

tokenizer = AutoTokenizer.from_pretrained(
    model_id,
    trust_remote_code=True
)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True
)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model.config.use_cache = False

print("Model loaded successfully")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100%                                                  967/967 [00:00<00:00, 30.3kB/s]

configuration_phi3.py:       11.2k/? [00:00<00:00, 289kB/s]

[transformers] A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- configuration_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, yo
tokenizer_config.json:       3.44k/? [00:00<00:00, 83.9kB/s]

tokenizer.json:       1.94M/? [00:00<00:00, 18.9MB/s]

tokenizer.model: 100%                                              500k/500k [00:01<00:00, 2.52MB/s]

added_tokens.json: 100%                                            306/306 [00:00<00:00, 8.66kB/s]

special_tokens_map.json: 100%                                      599/599 [00:00<00:00, 17.9kB/s]

modeling_phi3.py:       73.2k/? [00:00<00:00, 2.50MB/s]

[transformers] A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- modeling_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, yo
WARNING:transformers_modules.microsoft.Phi_hyphen_3_hyphen_mini_hyphen_4k_hyphen_instruct.f39ac1d28e925b323eae81227eaba4464caced
WARNING:transformers_modules.microsoft.Phi_hyphen_3_hyphen_mini_hyphen_4k_hyphen_instruct.f39ac1d28e925b323eae81227eaba4464caced
model.safetensors.index.json:       16.5k/? [00:00<00:00, 1.45MB/s]

Download complete: 100%      7.64G/7.64G [05:44<00:00, 210MB/s]

Fetching 2 files: 100%                                             2/2 [01:00<00:00, 60.86s/it]
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_4536/2171412925.py in <cell line: 0>()
     16 )
     17
---> 18 model = AutoModelForCausalLM.from_pretrained(
     19     model_id,
     20     quantization_config=bnb_config,

                                  6 frames
~/.cache/huggingface/modules/transformers_modules/microsoft/Phi_hyphen_3_hyphen_mini_hyphen_4k_hyphen_instruct/f39ac1d28e925b323
in _init_rope(self)
    294             )
    295         else:
--> 296             scaling_type = self.config.rope_scaling["type"]
    297             if scaling_type == "longrope":
    298                 self.rotary_emb = Phi3LongRoPEScaledRotaryEmbedding(self.head_dim, self.config)

KeyError: 'type'
```

Next steps: Explain error

```
import gc
import torch

try:
    del model
except:
    pass

gc.collect()
torch.cuda.empty_cache()
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, AutoConfig, BitsAndBytesConfig

model_id = "microsoft/Phi-3-mini-4k-instruct"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
```

```
)

tokenizer = AutoTokenizer.from_pretrained(
    model_id,
    trust_remote_code=True
)

config = AutoConfig.from_pretrained(
    model_id,
    trust_remote_code=True
)

print("Original rope_scaling:", config.rope_scaling)

# IMPORTANT FIX:
# Phi-3-mini-4k uses default RoPE here. Do not force longrope.
config.rope_scaling = None

print("Fixed rope_scaling:", config.rope_scaling)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    config=config,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
    attn_implementation="eager"
)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model.config.use_cache = False

print("Model loaded successfully")
```

```
Original rope_scaling: {'rope_theta': 10000.0, 'rope_type': 'default'}
Fixed rope_scaling: None
Loading weights: 100%                                                    195/195 [00:26<00:00,  6.72it/s]
generation_config.json: 100%                                                 181/181 [00:00<00:00, 10.8kB/s]
Model loaded successfully
```

```
from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model

model = prepare_model_for_kbit_training(model)

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj"
    ],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

```
trainable params: 8,912,896 || all params: 3,829,992,448 || trainable%: 0.2327
```

```
from transformers import TrainingArguments

OUTPUT_DIR = "/content/drive/MyDrive/phi3_agentic_orchestrator_lora_test"

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=1,
```

```
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=2e-4,
    num_train_epochs=1,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=200,
    save_total_limit=2,
    fp16=True,
    optim="paged_adamw_8bit",
    report_to="none",
    warmup_ratio=0.03,
    lr_scheduler_type="cosine"
)

print("Training arguments ready")
```

```
[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Training arguments ready
```

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    dataset_text_field="text",
    max_seq_length=2048,
    args=training_args,
)

print("Trainer ready")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_4536/3008340819.py in <cell line: 0>()
      1 from trl import SFTTrainer
      2
----> 3 trainer = SFTTrainer(
      4     model=model,
      5     tokenizer=tokenizer,

TypeError: SFTTrainer.__init__() got an unexpected keyword argument 'tokenizer'
```

Next steps: Explain error

```
from trl import SFTConfig, SFTTrainer

OUTPUT_DIR = "/content/drive/MyDrive/phi3_agentic_orchestrator_lora_test"

sft_config = SFTConfig(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=2e-4,
    num_train_epochs=1,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=200,
    save_total_limit=2,
    fp16=True,
    optim="paged_adamw_8bit",
    report_to="none",
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
    max_length=2048,
)

trainer = SFTTrainer(
    model=model,
    processing_class=tokenizer,
```

```
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    args=sft_config,
)

print("Trainer ready")
```

```
[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Adding EOS to train dataset: 100%                           950/950 [00:00<00:00, 4113.79 examples/s]
Tokenizing train dataset: 100%                              950/950 [00:02<00:00, 543.68 examples/s]
Adding EOS to eval dataset: 100%                            50/50 [00:00<00:00, 2330.04 examples/s]
Tokenizing eval dataset: 100%                               50/50 [00:00<00:00, 355.69 examples/s]
Trainer ready
```

```
trainer.train()
```

```
[transformers] `use_return_dict` is deprecated! Use `return_dict` instead!
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The attention mask API under
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:172: FutureWarning: The attention mask API unde
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:202: FutureWarning: The attention mask API unde
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
WARNING:transformers_modules.microsoft.Phi_hyphen_3_hyphen_mini_hyphen_4k_hyphen_instruct.f39ac1d28e925b323eae81227eaba4464caced
---------------------------------------------------------------------------
NotImplementedError                       Traceback (most recent call last)
/tmp/ipykernel_4536/4032920361.py in <cell line: 0>()
----> 1 trainer.train()

                                 7 frames
/usr/local/lib/python3.12/dist-packages/torch/amp/grad_scaler.py in _unscale_grads_(self, optimizer, inv_scale, found_inf,
allow_fp16)
    278             for device, per_dtype_grads in per_device_and_dtype_grads.items():
    279                 for grads in per_dtype_grads.values():
--> 280                     torch._amp_foreach_non_finite_check_and_unscale_(
    281                         grads,
    282                         per_device_found_inf.get(device),

NotImplementedError: "_amp_foreach_non_finite_check_and_unscale_cuda" not implemented for 'BFloat16'
```

Next steps: Explain error

```
import gc
import torch

try:
    trainer.model.zero_grad()
except:
    pass

gc.collect()
torch.cuda.empty_cache()

print("Cleared failed training state")
```

```
Cleared failed training state
```

```
bf16=False
fp16=True
```

```
from trl import SFTConfig, SFTTrainer

OUTPUT_DIR = "/content/drive/MyDrive/phi3_agentic_orchestrator_lora_test"

sft_config = SFTConfig(
    output_dir=OUTPUT_DIR,

    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,

    learning_rate=2e-4,
```

```
    num_train_epochs=1,

    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,

    save_steps=200,
    save_total_limit=2,

    # Safe fallback
    fp16=False,
    bf16=False,

    optim="paged_adamw_8bit",
    report_to="none",

    warmup_ratio=0.03,
    lr_scheduler_type="cosine",

    max_length=2048,
)

trainer = SFTTrainer(
    model=model,
    processing_class=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    args=sft_config,
)

print("Trainer rebuilt with fp16=False and bf16=False")
```

```
[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Adding EOS to train dataset: 100%                                                    950/950 [00:00<00:00, 8864.41 examples/s]
```